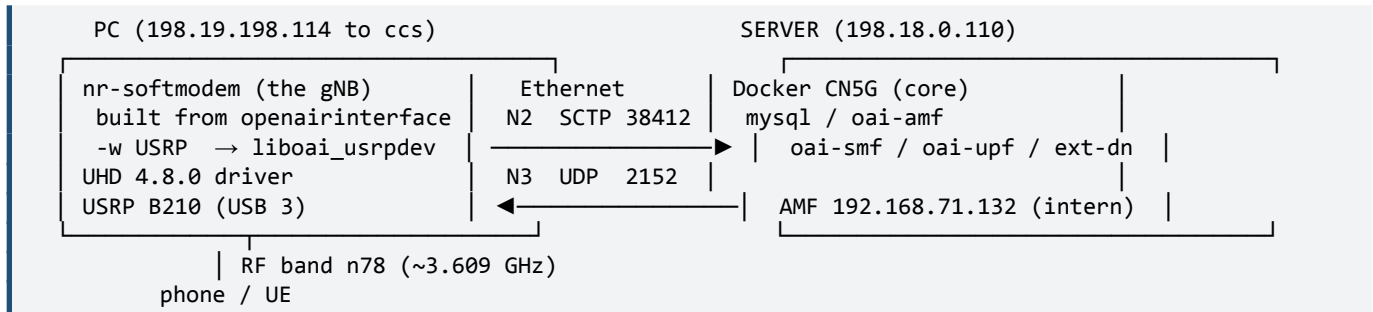


OAI 5G — USRP gNB (PC) + Core (Server)

Two-machine setup over Ethernet

A record of a split deployment: the 5G **core** runs in Docker on a **server**; the **gNB** runs natively on a **PC** driving a USRP B210. The two connect over the LAN. Covers what runs where, the exact IPs, the config edits, exposing the core, building the gNB, and the launch/verification.

1. Topology — what runs where



Key idea: **UHD is only the USRP driver**; the actual gNB is OAI’s nr-softmodem program, which must run on the PC (same machine as the radio) and talks NGAP to the AMF on the server.

Addresses used

- Server (core) LAN IP: 198.18.0.110
- PC (gNB) LAN IP: 198.19.198.114 (from `ip route get 198.18.0.110`)
- AMF internal Docker IP: 192.168.71.132 (not reachable from PC — see step 3)
- PLMN: MCC 001 / MNC 01 (must match SIM, gNB, core, and DB row)
- Band 78, 51 PRB (20 MHz), 30 kHz SCS, SSB $\approx$ 3.609 GHz

2. Server — the core

The Docker CN5G from the 5g_rfsimulator deployment is reused unchanged except for exposing two ports (step 3). Bring it up staged and wait for health:

```
cd ~/OAI_Begineers_Experiment/openairinterface5g/ci-scripts/yaml_files/5g_rfsimulator
docker compose up -d mysql oai-amf oai-smf oai-upf oai-ext-dn
docker compose ps          # mysql, oai-amf, oai-smf, oai-upf all (healthy)
```

Server OAI version (match on the PC): 2026.w24 (`git describe --tags`).

3. Expose the core to the PC (Option A: publish ports)

By default the AMF/UPF are only on the internal Docker network (192.168.71.x), which the PC cannot reach — `docker inspect` showed “38412/sctp”: null. Publish the N2 and N3 ports to the server host so the PC can reach them.

In the server’s `docker-compose.yaml`, add `ports:` to the two services:

```
oai-amf:
  ports:
    - "38412:38412/sctp"      # N2 / NGAP
oai-upf:
  ports:
    - "2152:2152/udp"        # N3 / GTP-U
```

Recreate the two containers and confirm the ports are published:

```
docker compose up -d --force-recreate oai-amf oai-upf
docker compose ps      # PORTS now show 0.0.0.0:38412->38412/sctp and 0.0.0.0:2152->2152/udp
sudo netstat -ltnp | grep 38412    # host now LISTENs on sctp 38412 (docker-proxy)
```

4. Verify reachability (PC → server)

```
# on the PC
ping -c 3 198.18.0.110          # 0% loss = LAN path OK
nc -vz -u 198.18.0.110 2152     # UPF N3 (UDP 2152) reachable = 'succeeded!'
ip route get 198.18.0.110       # 'src' = this PC's IP (198.19.198.114)
```

(SCTP 38412 can't be tested cleanly with plain nc; the server-side netstat LISTEN line from step 3 is the reliable proof.)

5. PC — install UHD (USRP driver)

UHD moves raw samples to/from the B210 over USB. Built from source (v4.8.0.0):

```
sudo apt install -y autoconf automake build-essential ccache cmake cpufrequtils doxygen ethtool g++
git inetutils-tools libboost-all-dev libncurses-dev libusb-1.0-0 libusb-1.0-0-dev libusb-dev
python3-dev python3-mako python3-numpy python3-requests python3-scipy python3-setuptools
python3-ruamel.yaml

git clone https://github.com/EttusResearch/uhd.git ~/uhd
cd ~/uhd
git checkout v4.8.0.0
cd host
mkdir build
cd build
cmake ../
make -j $(nproc)
make test # This step is optional
sudo make install
sudo ldconfig
sudo uhd_images_downloader
sudo uhd_find_devices
```

Detection confirmed: product: B210, type: b200. This proves the driver sees the hardware — it is **not** the gNB yet.

6. PC — build the OAI gNB (nr-softmodem)

The gNB is OAI's nr-softmodem, built with USRP support so it links to UHD. Match the server's version (2026.w24):

```
cd ~/OAI_gNB
git clone https://github.com/OPENAIRINTERFACE/openairinterface5g.git # gitlab SSH was blocked; used
GitHub mirror
cd openairinterface5g
git checkout 2026.w24 # same tag as the server
cd cmake_targets
./build_oai -I --install-optional-packages -w USRP # deps + mandatory B2xx TDD patch
./build_oai -w USRP --gNB # NOTE: omit --ninja (see gotcha)
ls -l ran_build/build/nr-softmodem # ~109 MB binary = the gNB
```

Gotcha (generator clash): the first build_oai -I configured with Unix Makefiles; adding --ninja afterwards fails with “Does not match the generator used previously”. Fix: drop --ninja, or rm -rf ran_build/build/CMakeCache.txt ran_build/build/CMakeFiles first, then use --ninja.

7. gNB config — the edits we made

File: ci-scripts/conf_files/gnb.sa.band78.51prb.usrbp200.conf on the PC (51 PRB / band 78 — a forgiving start for the B210 over USB 3). Changes:

7a. PLMN (match the SIM + core)

```
plmn_list = ({ mcc = 001; mnc = 01; mnc_length = 2;
               snssaiList = ({ sst = 1, sd = 0xffffffff }) });
```

7b. AMF address + gNB local interfaces (the IPs)

```
////////// AMF parameters:
amf_ip_address = ({ ipv4 = "198.18.0.110"; });      # the SERVER (core)

NETWORK_INTERFACES :
{
    GNB_IPV4_ADDRESS_FOR_NG_AMF = "198.19.198.114"; # this PC (N2)
    GNB_IPV4_ADDRESS_FOR_NGU    = "198.19.198.114"; # this PC (N3)
    GNB_PORT_FOR_S1U            = 2152;
};
```

Rule: **amf_ip_address** = the server; **GNB_IPV4_ADDRESS_FOR_*** = the machine running the gNB (the PC).

7c. USRP detection (RUs section)

```
RUs = ( {
    local_rf = "yes"; nb_tx = 1; nb_rx = 1;
    att_tx = 14; att_rx = 14; bands = [78];
    max_pdschReferenceSignalPower = -27; max_rxgain = 114;
    clock_src = "internal";
    sdr_addrs = "type=b200";      # <-- added: selects the B210
} );
```

7d. Core must match too

In the server's `mini_nonrf_config.yaml`, set the AMF `served_guami_list` and `plmn_support_list` to MCC 001 / MNC 01 so core, gNB, and SIM agree.

8. Launch the gNB and verify N2

```
cd ~/OAI_gNB/openairinterface5g/cmake_targets/ran_build/build
sudo ./nr-softmodem \
  -O ~/OAI_gNB/openairinterface5g/ci-scripts/conf_files/gnb.sa.band78.51prb.usrpb200.conf \
  --gNBs.[0].min_rxtxttime 6 -E --continuous-tx
```

SA mode is the default (no `--sa`). `-E --continuous-tx` are the required B210 flags.

Success markers seen in the gNB console:

- Send NGSetupRequest to AMF → Received NGSetupResponse from AMF (N2 up)
- Found USRP b200 / Detected Device: B210 / Operating over USB 3 (radio up)
- TX/RX tuned to 3.609 GHz, then RU 0 RF started and Frame.Slot stepping (live cell)

On the server, **while the gNB is running**:

```
docker logs rfsim5g-oai-amf 2>&1 | grep -A 6 "gNBs" | tail -10
# Status = Connected, Global Id 0x0E00, PLMN 001,01
```

Note: if you stop the gNB with Ctrl-C, the AMF table flips to Disconnected — that means “was connected, then stopped”, not a failure. Check the table while the gNB is still running to see Connected.

9. Outstanding before the phone attaches

The subscriber DB row must match the SIM and the PLMN. The row added had IMSI 999700000161730 (PLMN 999/70), but the gNB/core are PLMN 001/01. A 001/01 SIM will attach to the cell but fail authentication until the users row's IMSI starts 00101... and equals the SIM's real 15-digit IMSI.

Also, for real internet the UPF needs SNAT on and the host needs IP-forward + MASQUERADE for the UE subnet out its WAN interface (separate step).

10. One-glance recap

```
SERVER: core in Docker, expose 38412/sctp + 2152/udp, PLMN 001/01
PC:     UHD (driver) + nr-softmodem (gNB, -w USRP) + B210 on USB3
CONF:   amf_ip_address = server (198.18.0.110)
        GNB_IPV4_ADDRESS_FOR_NG_AMF/NGU = PC (198.19.198.114)
        plmn 001/01 ; sdr_addrs = type=b200 ; band 78, 51 PRB
LINK:   NGSetupRequest -> NGSetupResponse = N2 up = gNB Connected
```

11. Connecting the commercial UE — the PLMN-mismatch problem

The programmable SIM's IMSI starts 99970, so its SUCI carries **MCC 999 / MNC 70**, while the gNB and core are provisioned as **MCC 001 / MNC 01**. The AMF enforces a consistency check: it compares the PLMN inside the UE's SUCI against the gNB's configured PLMN, and on mismatch it sends a Registration Reject with cause PLMN not allowed and stops. That check is what blocked this SIM from registering.

Two ways to resolve it:

- (a) the clean way — make all four PLMNs agree by setting the gNB/core to 999/70 and provisioning the subscriber under that PLMN; or
- (b) the way used here — patch the AMF to skip the PLMN check. **We used (b).**

12. Custom AMF — removing the PLMN check

Clone the AMF source, comment out the PLMN-mismatch guard, rebuild the image, and point the core at it.

12a. The code edit

Repository: gitlab.eurecom.fr/oai/cn5g/oai-cn5g-amf · file: `src/amf-app/amf_n1.cpp`. Comment out this block ([line 1806](#)):

```
// if (imsi.mcc != gc->plmn.mcc || imsi.mnc != gc->plmn.mnc) {  
//     Logger::amf_n1().error(  
//         "PLMN (MCC %s, MNC %s) in SUCI does not match with gNB PLMN "  
//         "(MCC %s, MNC %s)",  
//         imsi.mcc, imsi.mnc, gc->plmn.mcc, gc->plmn.mnc);  
//     // Send Registration Reject with appropriate cause  
//     send_registration_reject_msg(  
//         ran_ue_ngap_id, amf_ue_ngap_id, k5gmmCausePlmnNotAllowed);  
//     cause = k5gmmCausePlmnNotAllowed;  
//     return false;  
// }
```

This removes the gatekeeper so the AMF accepts a 999/70 SUCI against the 001/01 gNB instead of rejecting it.

12b. Build the custom image

```
docker build --target oai-amf --tag oai-amf:custom -f docker/Dockerfile.amf.ubuntu .
```

12c. Point the core at the custom image

In the server's `docker-compose.yaml`, `oai-amf` service:

```
oai-amf:  
  container_name: "rfsim5g-oai-amf"  
  #image: oaisoftwarealliance/oai-amf:v2.2.1    # original (commented out)  
  image: oai-amf:custom                        # custom build
```

Then bring the core up staged as usual and launch the gNB (sections 2 and 8).

13. Result and caveats

Outcome: the gNB connected to the custom AMF, the commercial phone with the programmable SIM **registered**, the operator was selectable on the handset, and it **got connected**.

Verify

```
# on the server, while the phone is on:
docker logs rfsim5g-oai-amf 2>&1 | grep -iE "5GMM-REGISTERED|PDU Session" | tail -10
docker logs rfsim5g-oai-upf 2>&1 | tail -20
# on the phone: open a website / ping to confirm data
```

Caveats to remember

- This core runs a **modified AMF** with the PLMN check removed — it will accept UEs whose PLMN doesn't match the network. Fine for a controlled lab, but non-standard.
- Reverting to the stock oaisoftwarealliance/oai-amf:v2.2.1 image restores the check, and this 999/70 SIM would be rejected again.
- The "matching" alternative (set gNB + core + DB to 999/70) avoids patching the AMF and keeps the stack standard.
- For full internet (if not already): enable UPF SNAT, and on the server `sysctl -w net.ipv4.ip_forward=1` + a MASQUERADE rule for the UE subnet out the WAN interface.

14. Attach handshake and observations

When the commercial UE (SIM IMSI 999700000161730) powered on, it completed the full 5G attach over the air. The sequence below is the order it happened; each stage proceeds only if the previous one succeeded, and the PLMN/identity must stay consistent throughout.

UE (phone)	gNB (B210 / PC)	Core (server)
--RA: preamble----->		
<--Msg2 / Msg4-----		
--RRCSetupComplete-->	RRC_CONNECTED	
	--NGAP InitialUEMsg-->	create UE context
<==== NAS auth + security (DL/UL Info Transfer) =====>		(custom AMF: PLMN check skipped)
	<--Registration Accept--	5GMM-REGISTERED + GUTI
	--PDU Session Setup----->	IP allocated, GTP-U up
..... user data (GTP-U via UPF)		
	(after inactivity) -->	UP deactivation (idle)

Stage by stage — what to observe

- 1. Random Access (radio): [RAPROC] Initiating RA procedure with preamble 22; note estimated distance = real over-the-air timing advance.
- 2. RRC setup: Send RRC Setup → Received RRCSetupComplete (RRC_CONNECTED reached).
- 3. NGAP Initial UE Message: Selected PLMN in the NG Initial UE Message: MCC=001 MNC=01 → Create UE context for AMF.
- 4. NAS auth + security: gNB DL/UL Information Transfer exchanges carry the NAS messages; passes because the custom AMF skips the PLMN check.
- 5. Registration Accept (confirmed): AMF table shows IMSI 999700000161730 = 5GMM-REGISTERED with a GUTI.
- 6. PDU Session Establishment (confirmed set up): PDU_RES_SETUP_REQ for SUPI imsi-999700000161730; NAS PDU Session Identity 2 encoded; SMF allocates the UE IP.

- 7. User-plane / idle: GTP-U carries data via the UPF; after ~16 s inactivity, Trigger PDU Session UP Deactivation towards SMF — the session goes idle but stays registered, reactivating on new traffic.

The key thing to observe

The attach is a strict chain — **RA** → **RRC** → **NGAP** → **NAS** → **Registration** → **PDU session** → **user-plane** — where each step depends on the one before and on the PLMN/identity being consistent end to end. A failure at any stage stops the rest, so read the logs top-down to find where it stopped.

Observation on the PDU session (honest status)

A PDU session **was set up** (PDU_RES_SETUP_REQ + NAS PDU Session Identity for the SUPI). About 16 s later the AMF logged UP deactivation for several PDU sessions. This normally means the user-plane went **idle after inactivity** (CM-IDLE) — expected 5G behaviour, not a failure; the UE stays registered and the session reactivates when traffic resumes.

Sustained internet browsing was **not confirmed from these logs alone**. To confirm it: check the UE got an IP (SMF paa / ipv4 allocation) and enable the exit path — UPF SNAT on, plus on the server `sysctl -w net.ipv4.ip_forward=1` and a MASQUERADE rule for the UE subnet out the WAN interface (section 13 caveats).

```
# confirm IP allocation + user-plane on the server:
docker logs rfsim5g-oai-smf 2>&1 | grep -iE "paa|ipv4|allocat|99970000161730" | tail -20
docker logs rfsim5g-oai-upf 2>&1 | tail -30
# and simply: open a website / ping on the phone
```